



MODUL PERKULIAHAN

GETARAN MEKANIK

Kode Mata kuliah W132500020

Modul

Transformasi Fourier: Teori &

Abstrak

Pertemuan ini membahas transformasi sinyal getaran dari domain waktu ke domain frekuensi melalui Fourier Series,

Sub - CPMK

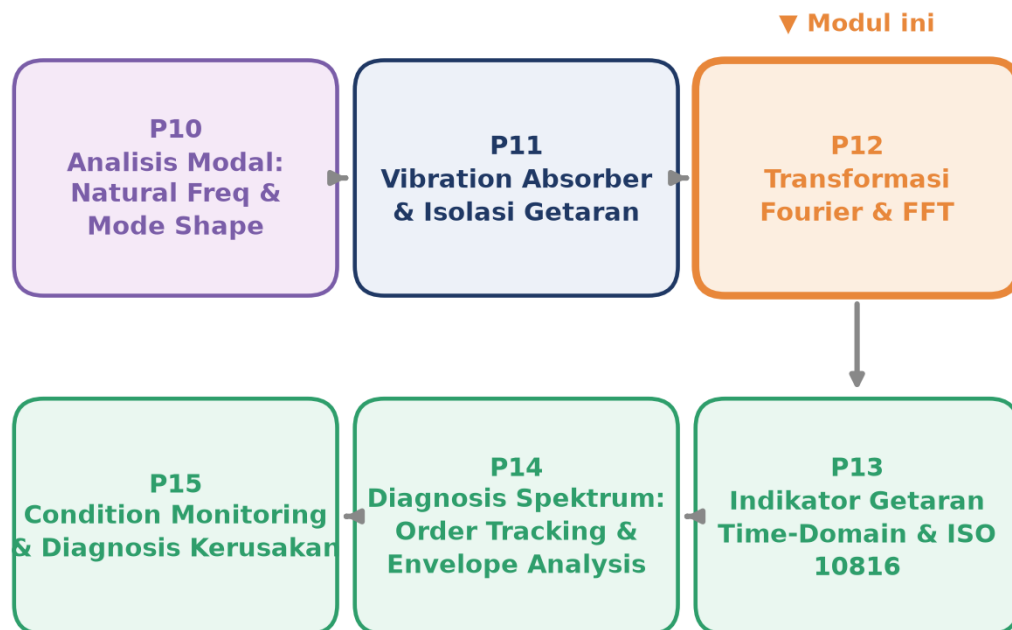
Sub-CPMK 6.1 – Menerapkan Konsep Transformasi Fourier

Disusun Oleh :
Dedik Romahadi, ST., M.Sc

🔗 Modul Interaktif: <https://dedik-romahadi.github.io/Mechanical-Engineering-Courses/Getaran-Mekanik/Modul/Modul-11.html>. Pada layar masuk, pilih tombol “👁 Mode Preview (Tanpa Login)” untuk melihat modul lengkap (animasi, kalkulator interaktif, editor kode Python langsung di browser) tanpa perlu akun. Soal pada Mode Preview bersifat tampilan saja; jawaban benar/salah dan poin tidak aktif.

PENDAHULUAN

Sinyal getaran yang direkam accelerometer atau proximity probe pada mesin industri hampir selalu berupa campuran banyak frekuensi karakteristik: putaran rotor, gear mesh, elemen bearing, dan noise lingkungan. Dilihat langsung sebagai gelombang waktu (waveform), komposisi frekuensi ini sulit dibedakan oleh mata karena semuanya tumpang tindih dalam satu kurva. Pertemuan ke-12 ini membahas Transformasi Fourier, yaitu alat matematis yang mengurai sinyal domain waktu $x(t)$ menjadi spektrum domain frekuensi $X(f)$, mengungkap komponen frekuensi yang tersembunyi di dalam waveform. Materi mencakup Fourier Series untuk sinyal periodik, Continuous Fourier Transform (CFT) untuk sinyal aperiodik, Discrete Fourier Transform (DFT) sebagai versi komputer dari CFT, algoritma Fast Fourier Transform (FFT) Cooley-Tukey yang mempercepat komputasi DFT secara drastis, teori sampling Nyquist beserta bahaya aliasing, dan windowing untuk mengatasi spectral leakage. Gambar 1 menunjukkan posisi Pertemuan 12 dalam keseluruhan alur mata kuliah Getaran Mekanik.



Gambar 1. Posisi Pertemuan 12 (Transformasi Fourier dan FFT) dalam alur mata kuliah Getaran Mekanik, menjembatani analisis modal/isolasi getaran (P10-P11) dengan indikator dan diagnosis spektrum (P13-P15).

Materi ditutup dengan implementasi Python `numpy.fft` dan `scipy.fft` di Jupyter Notebook, tugas terstruktur berbasis parameter numerik, dan studi kasus diagnosis spektrum motor-gearbox industri yang menuntut penerapan seluruh konsep secara terpadu.

Capaian Pembelajaran (Sub-CPMK)

- **Sub-CPMK 6.1:** mampu menerapkan konsep Transformasi Fourier untuk mengurai sinyal getaran domain waktu menjadi spektrum domain frekuensi, memilih parameter akuisisi (sample rate, jumlah sample, window function) yang tepat, dan menginterpretasi hasil FFT untuk diagnosis kondisi mesin (CPMK 6).
- **Setelah pertemuan ini mahasiswa dapat:** menjelaskan hubungan Fourier Series, Continuous Fourier Transform, DFT, dan FFT sebagai satu keluarga transformasi yang berevolusi dari sinyal periodik-kontinu menuju sinyal diskrit-berhingga; menerapkan Nyquist Theorem untuk mencegah aliasing; menerapkan windowing untuk mengurangi spectral leakage; serta mengimplementasikan FFT dengan `numpy/scipy` untuk mendiagnosis komponen frekuensi karakteristik kerusakan mesin.

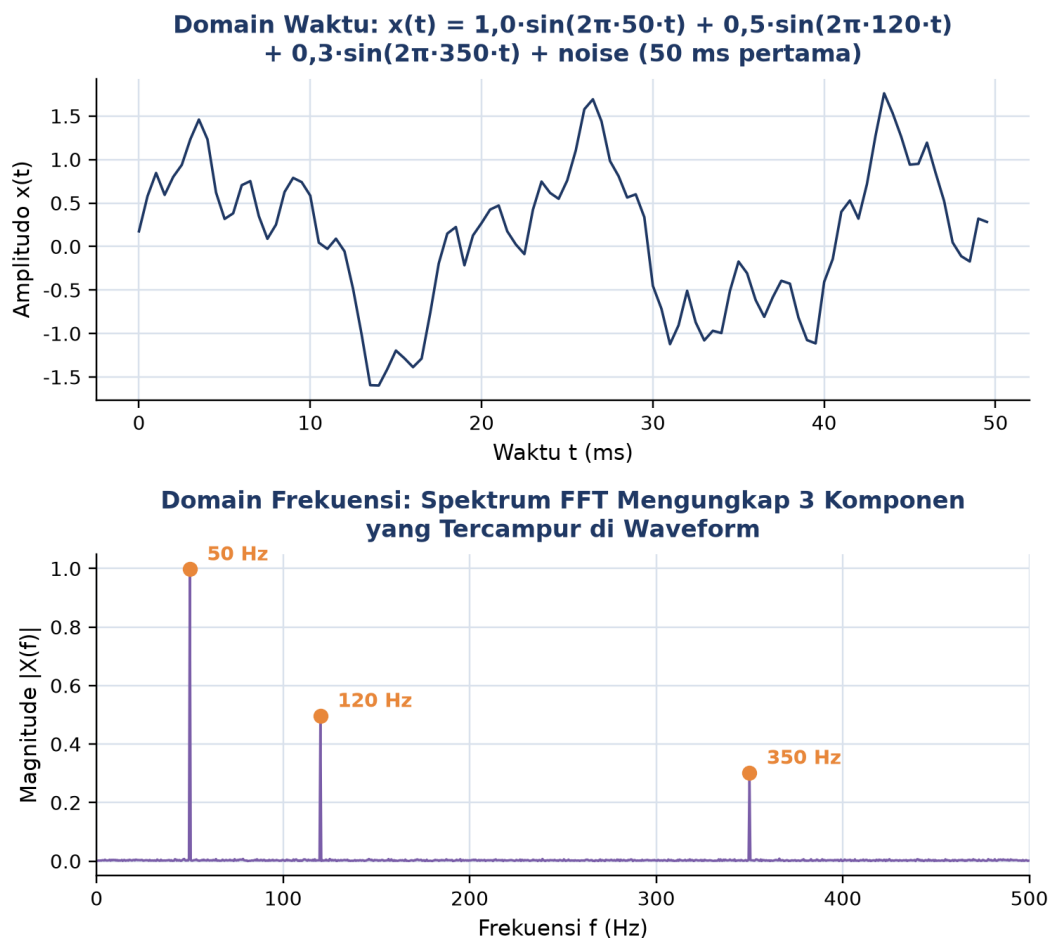
DOMAIN WAKTU, DOMAIN FREKUENSI, DAN FOURIER SERIES

Dua Sisi Sinyal yang Sama: Sinyal getaran mesin direkam sebagai $x(t)$, yaitu amplitudo terhadap waktu. Representasi ini disebut domain waktu (time domain): bagus untuk melihat shock, transient, dan perubahan amplitudo, tetapi sulit dibaca komposisi frekuensinya bila banyak komponen tercampur. Transformasi Fourier mengurai sinyal domain waktu menjadi spektrum frekuensi, yaitu peta magnitude $|X(f)|$ terhadap frekuensi f : setiap puncak pada spektrum merepresentasikan satu komponen sinusoidal penyusun sinyal asli. Kedua representasi setara, mengandung informasi yang sama, hanya berbeda sudut pandang; Fourier Transform (FT) mengubah waktu menjadi frekuensi, Inverse Fourier Transform (IFT) mengembalikannya.

$$x(t) = A_1 \sin(\omega_1 t) + A_2 \sin(\omega_2 t) + \dots \leftrightarrow |X(f)| \text{ pada } f_1, f_2, \dots$$

Pentingnya representasi frekuensi ini sangat terasa di diagnosis getaran mesin: bearing rusak menghasilkan frekuensi karakteristik (BPFO, BPFI, BSF), gear mesh muncul di kelipatan $N \times \text{rpm}$, unbalance di $1 \times \text{rpm}$, misalignment umumnya di $2 \times \text{rpm}$. Tanpa Fourier, seluruh komponen ini bercampur di waveform dan tidak dapat didiagnosis; itulah sebabnya lebih dari 90% pekerjaan vibration analysis di industri dilakukan di domain frekuensi. Gambar 2 memperlihatkan langsung perbedaan kedua representasi pada sinyal sintesis tiga komponen (50, 120, dan 350 Hz) yang tercampur dengan noise: waveform domain

waktu tampak rumit dan sulit dibaca, sedangkan spektrum FFT di bawahnya menunjukkan tiga puncak tajam yang persis mengungkap ketiga frekuensi penyusunnya.



Gambar 2. Sinyal sintesis tiga komponen frekuensi (50, 120, 350 Hz) plus noise: domain waktu (atas) tampak rumit, domain frekuensi/spektrum FFT (bawah) mengungkap ketiga puncak dengan jelas.

Interpretasi Gambar 2: Gambar 2 menunjukkan bahwa ketiga komponen frekuensi yang sama sekali tidak terlihat sebagai pola berulang jelas pada waveform domain waktu, langsung tampak sebagai tiga puncak terpisah pada spektrum domain frekuensi, masing-masing pada 50 Hz (amplitudo terbesar), 120 Hz, dan 350 Hz (amplitudo terkecil), sesuai proporsi amplitudo aslinya (1,0 : 0,5 : 0,3). Inilah kekuatan mendasar Transformasi Fourier: mengubah masalah pengenalan pola visual yang sulit menjadi pembacaan puncak sederhana.

Fourier Series: Joseph Fourier (1822) menemukan bahwa setiap sinyal periodik dengan periode T dapat dinyatakan sebagai jumlah tak hingga sinusoidal berfrekuensi kelipatan frekuensi dasar $f_0 = 1/T$. Temuan ini, Fourier Series, adalah fondasi seluruh transformasi Fourier modern.

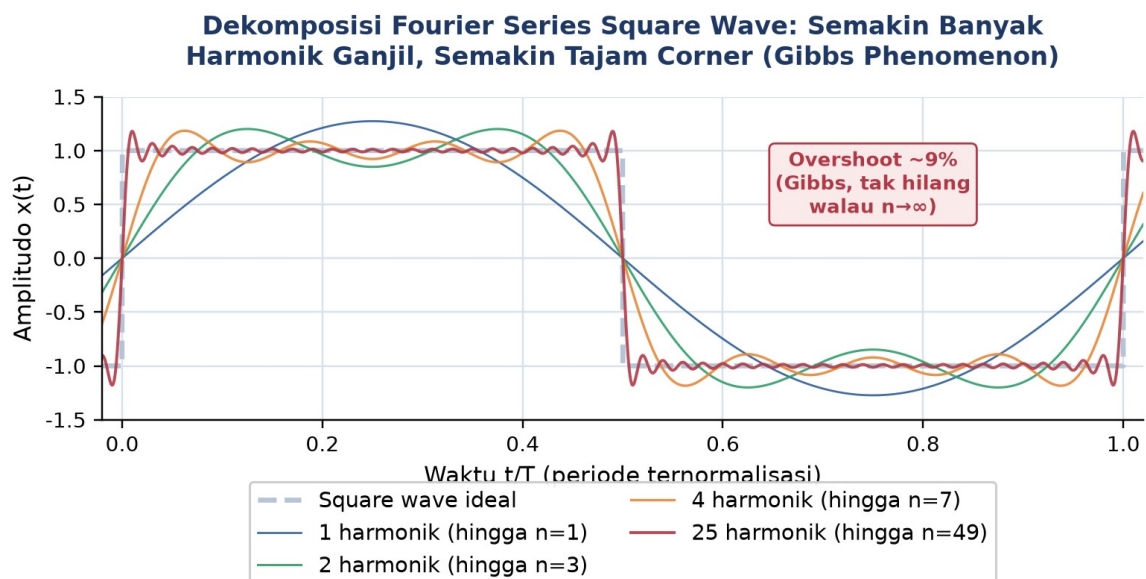
$$x(t) = a_0/2 + \sum [a_n \cos(2\pi n f_0 t) + b_n \sin(2\pi n f_0 t)], \quad n = 1, 2, 3, \dots$$

- **Frekuensi Dasar:** $f_0 = 1/T$ adalah frekuensi terendah dalam dekomposisi; komponen lain selalu kelipatan integer f_0 , $2f_0$, $3f_0$, dan seterusnya, disebut harmonik (f_0 = harmonik 1, $2f_0$ = harmonik 2, dst).
- **Koefisien Fourier:** $a_n = (2/T) \int_0^T x(t) \cos(2\pi n f_0 t) dt$ dan $b_n = (2/T) \int_0^T x(t) \sin(2\pi n f_0 t) dt$, dihitung atas satu periode T ; amplitudo komponen ke- n adalah $A_n = \sqrt{a_n^2 + b_n^2}$.
- **Bentuk Eksponensial Kompleks:** $x(t) = \sum C_n e^{j2\pi n f_0 t}$ dengan $C_n = (1/T) \int_0^T x(t) e^{-j2\pi n f_0 t} dt$; bilangan kompleks C_n menggabung amplitudo dan fase dalam satu nilai, formulasi yang kemudian digeneralisasi ke CFT dan FFT.

Contoh Konkret: Dekomposisi Square Wave. Contoh klasik dekomposisi Fourier Series adalah square wave, yang tersusun hanya dari harmonik ganjil dengan amplitudo berbanding terbalik dengan nomor harmoniknya.

$$x(t) = (4/\pi) [\sin(\omega_0 t) + \sin(3 \omega_0 t)/3 + \sin(5 \omega_0 t)/5 + \dots]$$

Semakin banyak harmonik ganjil dijumlahkan, semakin tajam sudut (corner) square wave yang terbentuk, tetapi tetap muncul overshoot sekitar 9% tepat di sekitar diskontinuitas yang tidak pernah hilang berapa pun banyaknya harmonik ditambahkan; fenomena ini disebut Gibbs Phenomenon, ditemukan oleh J. Willard Gibbs pada 1899. Gambar 3 memperlihatkan proses ini secara bertahap: kurva 1 harmonik masih berupa sinusoida murni, kurva 25 harmonik (hingga $n=49$) sudah sangat mendekati bentuk square wave ideal, tetapi overshoot di dekat setiap tepi tetap bertahan pada besaran yang hampir konstan.



Gambar 3. Dekomposisi Fourier Series square wave dengan jumlah harmonik ganjil bertambah (1, 3, 7, dan 25 harmonik); semakin banyak harmonik semakin tajam corner, tetapi overshoot Gibbs sekitar 9% tetap bertahan.

Spektrum Diskrit sebagai Sidik Jari Mesin: Spektrum sinyal periodik selalu diskrit, yaitu hanya bernilai non-zero pada frekuensi kelipatan f_0 ; ini berbeda dengan sinyal aperiodik yang spektrumnya kontinu (dibahas pada Bagian selanjutnya). Ciri khas spektrum mesin dalam kondisi steady-state adalah garis-garis tajam pada f_0 dan harmoniknya, sebuah pola teratur yang menjadi penanda mesin sehat. Aturan diagnosis cepat yang berlaku umum: sinyal getaran mesin steady-state (rotor berputar konstan) selalu periodik dengan periode sama dengan satu putaran shaft, sehingga $f_0 = \text{rpm}/60$; harmonik 2x sering menandakan misalignment, sementara harmonik 3x dan lebih tinggi umumnya menandakan looseness atau impulsive defect. Pemahaman Fourier Series sebagai konsep awal ini menjadi syarat wajib sebelum mempelajari FFT pada Bagian berikutnya.

CONTINUOUS FOURIER TRANSFORM DAN DISCRETE FOURIER TRANSFORM

Generalisasi ke Sinyal Aperiodik: Fourier Series hanya berlaku untuk sinyal periodik. Untuk sinyal aperiodik seperti transient, single pulse, atau sinyal terbatas waktu, solusinya adalah mengambil limit T menuju tak hingga dari Fourier Series, menghasilkan Continuous Fourier Transform (CFT), yaitu integral kontinu yang menghasilkan spektrum frekuensi kontinu (bukan lagi garis-garis diskrit).

$$\text{Forward: } X(f) = \text{integral}(-\text{tak hingga, } +\text{tak hingga}) x(t) e^{-j2\pi f t} dt$$

$$\text{Inverse: } x(t) = \text{integral}(-\text{tak hingga, } +\text{tak hingga}) X(f) e^{+j2\pi f t} df$$

- **Spektrum Kontinu:** $X(f)$ didefinisikan untuk semua f real (positif dan negatif), tidak lagi terbatas pada kelipatan f_0 ; magnitudo $|X(f)|$ menunjukkan kerapatan energi spektral pada frekuensi f .
- **Trade-off Pulse-Spektrum:** pulse waktu pendek (sharp spike) memiliki spektrum lebar, energi tersebar ke banyak frekuensi; pulse panjang sebaliknya lebih sempit di spektrum. Hubungan ini adalah uncertainty principle: $\Delta t \times \Delta f$ lebih besar atau sama dengan suatu konstanta, sempit di waktu selalu berarti lebar di frekuensi.
- **Sifat Linearitas:** FT bersifat linear, $\text{FT}[a x(t) + b y(t)] = a X(f) + b Y(f)$, sehingga spektrum sinyal gabungan sama dengan jumlah spektrum masing-masing komponen; sifat inilah yang membuat diagnosis getaran bekerja karena tiap kerusakan dapat diidentifikasi terpisah di spektrum walau bercampur di waveform.

Dari CFT ke DFT: Versi Komputer. CFT adalah definisi matematis murni; komputer tidak dapat menghitung integral dari minus tak hingga sampai plus tak hingga. Untuk implementasi nyata dibutuhkan Discrete Fourier Transform (DFT), versi diskrit yang bekerja pada sinyal sampled/discrete-time. Sensor accelerometer mengeluarkan sample diskrit $x[n]$

$= x(n \Delta t)$ untuk $n = 0, 1, \dots, N-1$, dan DFT mentransformasi N sample domain waktu menjadi N nilai domain frekuensi.

$$X[k] = \text{Sigma}(n=0..N-1) x[n] e^{(-j*2*\pi*k*n/N)}, \quad k = 0, 1, \dots, N-1$$

- **Indeks k sebagai Frequency Bin:** setiap nilai $X[k]$ berkaitan dengan frekuensi $f[k] = k f_s/N$, dengan $f_s = 1/\Delta t$ adalah sample rate; resolusi frekuensi $\Delta f = f_s/N$, semakin banyak sample N maka resolusi semakin halus.
- **Symmetry untuk Sinyal Real:** bila $x[n]$ real (umumnya berlaku untuk sinyal getaran), maka $X[N-k] = X^*[k]$ (conjugate); akibatnya hanya $N/2 + 1$ nilai yang independen, sisanya redundant, sehingga plot magnitude FFT biasanya hanya ditampilkan dari 0 sampai $N/2$.
- **Inverse DFT (IDFT):** mengembalikan domain waktu dari domain frekuensi: $x[n] = (1/N) \text{Sigma} X[k] e^{(+j*2*\pi*k*n/N)}$; kunci perbedaannya dengan DFT adalah tanda eksponen positif (bukan negatif) dan faktor pembagi $1/N$.

Contoh Numerik: Bin Width dan Frekuensi Maksimum. Sebagai ilustrasi numerik konkret, untuk sample rate $f_s = 25.600$ Hz dan $N = 4.096$ sample (satu buffer akuisisi), bin width $\Delta f = 25.600/4.096 = 6,25$ Hz per bin; frekuensi maksimum yang dapat direkam adalah $f_s/2 = 12.800$ Hz (batas Nyquist); total 4.096 bin, dengan bin 0 sebagai komponen DC dan bin 1 sampai 2.048 sebagai frekuensi positif yang berguna. Bila resolusi lebih halus dibutuhkan (misalnya 1 Hz), waktu rekam harus diperpanjang atau N diperbesar, dengan konsekuensi kebutuhan memori yang lebih besar.

Trade-off Klasik Resolusi Frekuensi vs Waktu: Trade-off ini bersifat mendasar dan tidak dapat "dicurangi": resolusi frekuensi Δf berbanding terbalik dengan resolusi waktu, $\Delta f \times T = 1$, dengan $T = N \times \Delta t$ adalah durasi total rekaman. Untuk mendapatkan resolusi 1 Hz dibutuhkan rekaman 1 detik penuh; untuk resolusi 0,1 Hz dibutuhkan 10 detik. Inilah konsekuensi langsung dari uncertainty principle yang sudah disinggung pada CFT.

Masalah Kompleksitas $O(N^2)$: DFT naive menghitung setiap $X[k]$ dengan N perkalian, sehingga total operasi adalah $N \times N = N^2$. Untuk $N = 1.024$, ini sudah mencapai 1 juta operasi; untuk $N = 65.536$, mencapai 4 milyar operasi, memakan waktu beberapa detik pada komputer biasa. Kompleksitas $O(N^2)$ inilah yang memotivasi pengembangan algoritma Fast Fourier Transform pada bagian berikutnya.

Tabel 1 merangkum keempat anggota keluarga transformasi Fourier beserta domain sinyal, bentuk spektrum, dan kompleksitas komputasinya, memperlihatkan evolusi dari Fourier Series (sinyal periodik) hingga FFT (implementasi cepat DFT pada komputer).

Tabel 1. Perbandingan keluarga transformasi Fourier: domain sinyal, bentuk spektrum, dan kompleksitas komputasi.

Transformasi	Domain Sinyal	Bentuk Spektrum	Kompleksitas
Fourier Series	Periodik, kontinu	Diskrit (garis pada kelipatan f_0)	Analitik (integral)
Continuous FT (CFT)	Aperiodik, kontinu	Kontinu (semua f real)	Analitik (integral)
Discrete FT (DFT)	Aperiodik, diskrit (N sample)	Diskrit (N nilai kompleks)	$O(N^2)$, naive
Fast FT (FFT)	Aperiodik, diskrit (N sample)	Diskrit (identik dengan DFT)	$O(N \log^2 N)$, Cooley-Tukey

FAST FOURIER TRANSFORM: ALGORITMA COOLEY-TUKEY

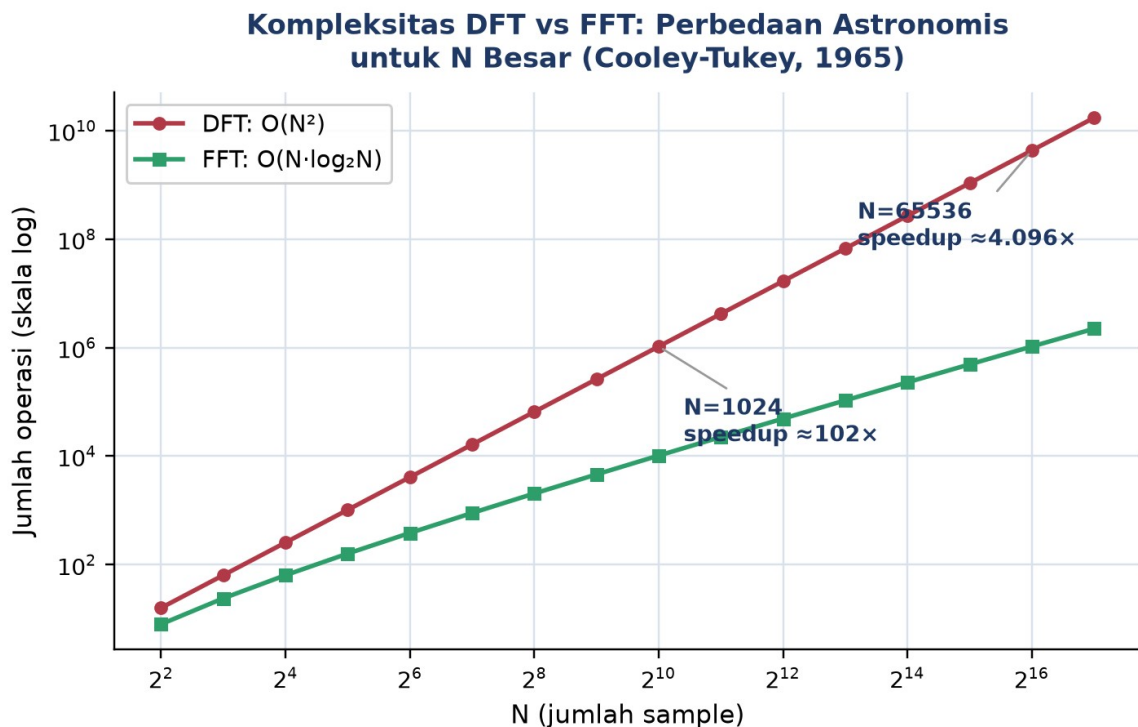
Tahun 1965, James Cooley dan John Tukey mempublikasikan algoritma yang mempercepat komputasi DFT dari $O(N^2)$ menjadi $O(N \log^2 N)$, disebut Fast Fourier Transform (FFT). Untuk $N = 1.024$, jumlah operasi turun dari sekitar 1 juta menjadi sekitar 10 ribu, sekitar 100 kali lebih cepat; untuk N besar, perbedaannya menjadi sangat signifikan. FFT membuka era pemrosesan sinyal real-time, format kompresi audio/gambar modern, dan vibration analyzer digital yang dipakai luas di industri saat ini.

DFT: $O(N^2)$ FFT: $O(N \log^2 N)$ Speedup teoretis proporsional dengan $N/\log^2 N$

Gambar 4 memvisualisasikan perbandingan kompleksitas ini pada skala logaritmik: untuk N kecil kedua kurva masih berdekatan, tetapi untuk N besar (mendekati $2^{16} = 65.536$) jarak keduanya menjadi sangat lebar, mengonfirmasi mengapa DFT naive praktis tidak terpakai lagi untuk N besar di aplikasi nyata.

Contoh Konkret: Dampak Kecepatan FFT pada Monitoring Real-Time. Sebagai ilustrasi konkret bagaimana perbedaan kompleksitas ini berdampak langsung pada kelayakan sistem monitoring real-time, bayangkan sebuah vibration analyzer portabel berbasis mikrokontroler embedded (mis. ARM Cortex-M7 pada 400 MHz) yang harus memproses buffer $N = 8.192$ sample setiap detik untuk menghasilkan spektrum FFT secara kontinu. Dengan DFT naive $O(N^2)$, jumlah operasi yang dibutuhkan mencapai sekitar $8.192^2 = 67$ juta operasi kompleks per buffer; pada prosesor embedded dengan throughput floating-point terbatas, ini bisa memakan waktu ratusan milidetik hingga lebih dari satu detik per buffer, jauh melampaui anggaran waktu satu detik yang tersedia sehingga sistem tidak mampu mengikuti laju akuisisi data secara real-time (backlog buffer menumpuk). Dengan algoritma FFT Cooley-Tukey $O(N \log^2 N)$, jumlah operasi turun menjadi sekitar $8.192 \times 13 = 106.496$ operasi, sekitar 630 kali lebih sedikit, sehingga waktu komputasi turun ke orde satu hingga beberapa milidetik saja per buffer, jauh di bawah anggaran waktu satu detik. Perbedaan inilah yang secara praktis menentukan apakah sebuah vibration analyzer

mampu menampilkan spektrum secara real-time (update tiap detik atau lebih cepat) atau hanya mampu bekerja secara batch/offline; hampir seluruh perangkat vibration analyzer komersial saat ini mengandalkan implementasi FFT (bukan DFT naive) tepat karena alasan kelayakan komputasi real-time semacam ini.



Gambar 4. Kompleksitas DFT $O(N^2)$ dibandingkan FFT $O(N \log_2 N)$ pada skala log-log; perbedaan jumlah operasi menjadi sangat besar seiring N membesar.

- **Divide-and-Conquer:** FFT membagi N-point DFT menjadi dua sub-masalah N/2-point DFT, satu untuk sample bernomor genap dan satu untuk sample bernomor ganjil, lalu menggabungkan hasilnya memakai twiddle factor; proses ini diulang secara rekursif hingga ukuran sub-DFT tinggal 2, menghasilkan total $\log_2 N$ lapisan komputasi.
- **Twiddle Factor:** faktor rotasi $W_N^k = e^{(-j \cdot 2\pi \cdot k/N)}$ yang menggabungkan dua sub-DFT; nilainya dihitung sekali di awal lalu disimpan dalam tabel (lookup table), strategi klasik menukar memori dengan kecepatan komputasi, dikenal sebagai operasi butterfly.
- **Syarat $N = 2^k$:** algoritma Cooley-Tukey radix-2 paling efisien ketika N adalah pangkat dua (256, 512, 1024, dan seterusnya); untuk N lain tersedia algoritma Bluestein, Rader, atau mixed-radix, yang sudah ditangani otomatis oleh implementasi numpy.fft dan scipy.fft tanpa mahasiswa perlu memilih manual.

Dampak Industri FFT: Dampak FFT pada dunia teknologi modern sangat luas: tanpa FFT tidak akan ada vibration analyzer real-time untuk predictive maintenance, tidak ada

kompresi audio MP3/AAC (memakai transformasi kerabat FFT), tidak ada kompresi gambar JPEG (memakai Discrete Cosine Transform, kerabat dekat FFT), tidak ada komunikasi 4G/5G berbasis modulasi OFDM (membagi spektrum menjadi banyak sub-carrier), dan tidak ada rekonstruksi citra MRI (k-space reconstruction). Algoritma yang dipublikasikan tahun 1965 ini menjadi salah satu algoritma paling berpengaruh dalam sejarah komputasi.

Implementasi Praktis: Mahasiswa tidak perlu menulis implementasi FFT dari nol karena sudah tersedia library matang dan teruji: `numpy.fft.fft(x)` mengembalikan array kompleks berisi spektrum penuh (frekuensi positif dan negatif); `numpy.fft.rfft(x)` khusus untuk input real, mengembalikan hanya setengah spektrum sehingga lebih efisien; `scipy.fft.fft(x)` adalah versi `scipy` yang umumnya sedikit lebih cepat untuk array besar. Yang perlu dipahami mahasiswa adalah cara interpretasi output: frequency bins, dan perbedaan antara magnitudo mentah dengan magnitudo yang sudah dinormalisasi ke satuan amplitudo asli.

Konversi Output FFT ke Amplitudo Fisik: Output $X = \text{np.fft.fft}(x)$ berisi N nilai kompleks. Magnitude per bin dihitung sebagai $\text{mag} = \text{np.abs}(X) / N$, dengan normalisasi $1/N$ agar diperoleh satuan amplitudo asli; sedangkan frekuensi per bin diperoleh dari $f = \text{np.fft.fftfreq}(N, d=1/\text{fs})$. Untuk keperluan plotting, gunakan hanya separuh pertama array $f[:N//2]$ dan $\text{mag}[:N//2]$, mengabaikan bagian mirror yang redundant sesuai sifat symmetry sinyal real.

SAMPLE RATE, NYQUIST, DAN WINDOWING

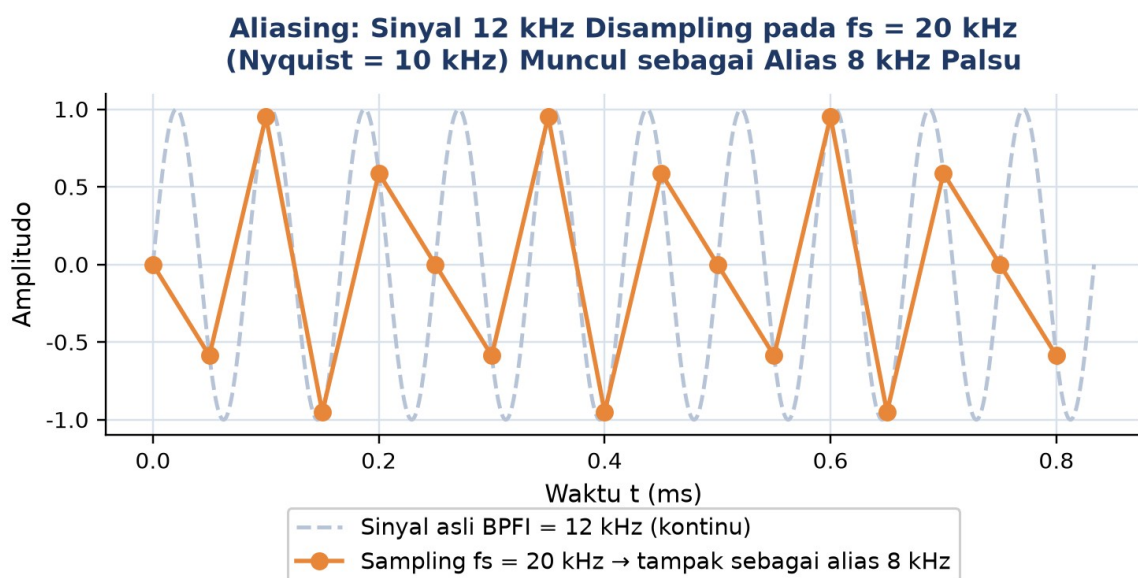
Mengapa Tidak Boleh Sampling Terlalu Lambat: ADC pada sensor getaran mengambil sample diskrit pada laju f_s (Hz). Pertanyaannya: berapa f_s minimum agar sinyal dengan frekuensi tertinggi f_{max} dapat direkam tanpa distorsi? Jawabannya adalah Nyquist Sampling Theorem, yaitu sample rate minimum harus dua kali frekuensi tertinggi yang ingin direkam. Bila dilanggar, terjadi aliasing, yaitu frekuensi tinggi yang menyamar menjadi frekuensi rendah palsu pada spektrum hasil.

$$f_s \geq 2 \times f_{\text{max}} \quad \leftrightarrow \quad f_{\text{max}} \leq f_s/2 \text{ (Nyquist frequency)}$$

- **Nyquist Frequency = $f_s/2$:** frekuensi maksimum yang dapat direkam akurat sama dengan setengah sample rate; untuk $f_s = 25.600$ Hz, $f_{\text{max}} = 12.800$ Hz. Spektrum FFT ditampilkan dari 0 sampai $f_s/2$, disebut Nyquist range.
- **Aliasing: Frekuensi Palsu:** sinyal pada f lebih besar dari $f_s/2$ akan terlipat balik dan muncul sebagai frekuensi palsu $f_{\text{alias}} = |f - k f_s|$; contoh, sinyal 13 kHz disampling pada $f_s = 20$ kHz akan muncul sebagai 7 kHz palsu di spektrum, sebuah kesalahan diagnosis yang berbahaya.

- **Anti-Aliasing Filter (AAF):** filter lowpass analog dipasang sebelum ADC untuk memotong sinyal di atas $f_s/2$; vibration analyzer industri selalu dilengkapi anti-aliasing filter (AAF) built-in, biasanya dengan cutoff sekitar $f_s/2,5$ untuk memberi margin roll-off realistis.
- **Pemilihan f_s Praktis:** aturan praktis industri untuk getaran mesin adalah $f_s = 2,56 \times f_{\max}$ target (bukan sekadar $2x$); faktor 2,56 memberi margin untuk roll-off filter yang tidak ideal secara fisik. Bila analisis dibutuhkan hingga 10 kHz (bearing frekuensi tinggi), maka f_s harus minimal 25,6 kHz.

Studi Kasus: Aliasing BPFI 12 kHz pada $f_s = 20$ kHz. Sebagai ilustrasi konsekuensi nyata aliasing, bayangkan sebuah bearing dengan BPFI (Ball Pass Frequency Inner Race) sebesar 12 kHz. Bila sistem akuisisi memakai $f_s = 20$ kHz, maka Nyquist = 10 kHz, dan sinyal 12 kHz akan alias ke $(20 - 12) = 8$ kHz, sehingga muncul sebagai komponen 8 kHz palsu pada spektrum. Tim maintenance yang tidak menyadari hal ini bisa salah mendiagnosis "ada puncak 8 kHz", padahal sumber sebenarnya adalah komponen 12 kHz yang jauh berbeda maknanya; kesalahan diagnosis semacam ini berpotensi mengarah pada keputusan perawatan yang salah dan biaya yang sia-sia. Solusinya adalah menaikkan f_s ke minimal 30 kHz ke atas, atau memasang AAF dengan cutoff di bawah 10 kHz agar komponen 12 kHz sudah tersaring sebelum masuk ADC. Gambar 5 memvisualisasikan mekanisme aliasing ini: sinyal kontinu 12 kHz yang disampling pada $f_s = 20$ kHz menghasilkan titik-titik sample yang, bila dihubungkan, membentuk gelombang 8 kHz yang jelas berbeda dari sinyal aslinya.



Gambar 5. Sinyal BPFI 12 kHz (kontinu, garis putus-putus) disampling pada $f_s = 20$ kHz (Nyquist = 10 kHz), menghasilkan titik-titik sample yang membentuk alias 8 kHz palsu.

Windowing: Mengatasi Spectral Leakage. DFT/FFT mengasumsikan sinyal bersifat periodik sepanjang buffer N sample. Pada praktiknya, buffer akuisisi hampir tidak pernah persis berisi jumlah periode bulat, sehingga tepi awal dan akhir buffer tidak menyambung mulus. Akibatnya, energi "bocor" ke bin-bin frekuensi di sekitar puncak sebenarnya, mengaburkan diagnosis; fenomena ini disebut spectral leakage. Solusinya adalah mengalikan sinyal dengan window function sebelum FFT untuk meratakan tepi buffer menuju nol secara bertahap.

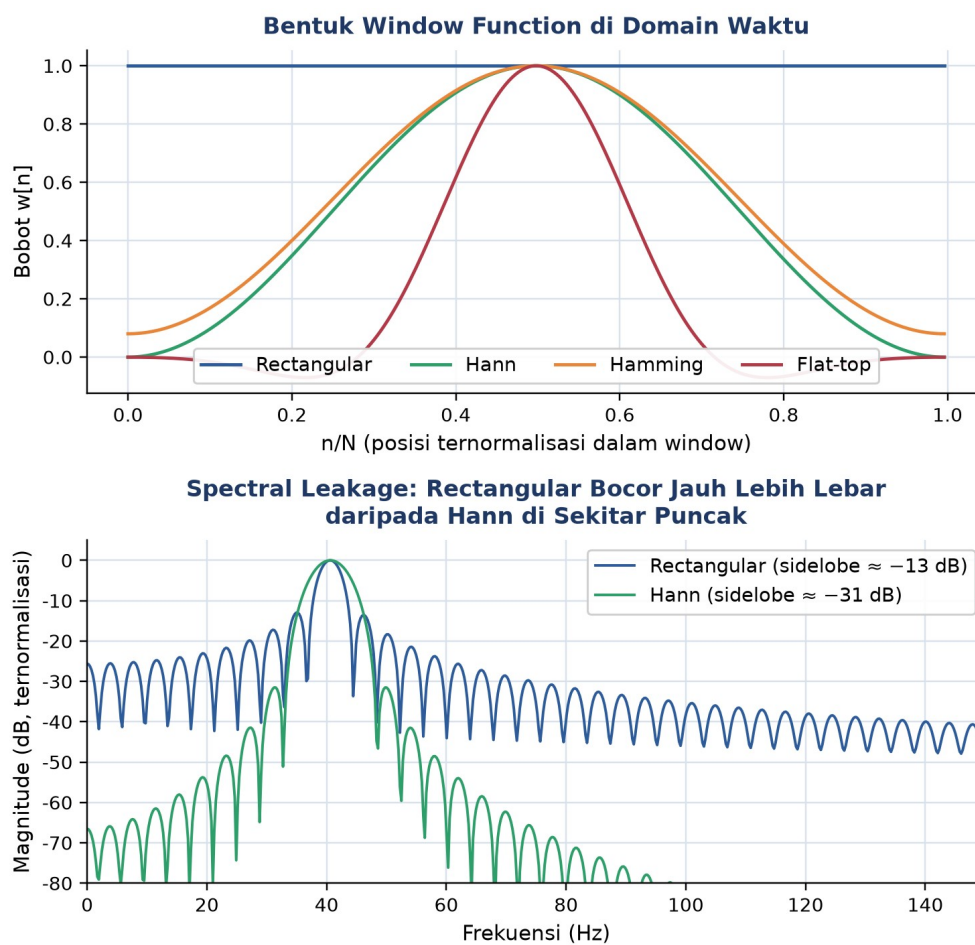
$$x_windowed[n] = x[n] \times w[n] \rightarrow X_windowed = FFT(x_windowed)$$

- **Rectangular (Tanpa Window):** $w[n] = 1$ untuk semua n , tidak meratakan tepi sama sekali sehingga leakage paling besar (side-lobe sekitar -13 dB); hanya cocok untuk sinyal transient yang secara alami sudah nol di tepi buffer, misalnya hammer impact test.
- **Hann (Hanning):** $w[n] = 0,5 (1 - \cos(2 \pi n/N))$, berbentuk lonceng sederhana; default umum untuk sinyal getaran rotor steady-state, memberi trade-off yang baik antara resolusi frekuensi dan penekanan side-lobe (sekitar -31 dB).
- **Hamming:** $w[n] = 0,54 - 0,46 \cos(2 \pi n/N)$, mirip Hann tetapi tepinya tidak persis nol; side-lobe lebih rendah (sekitar -43 dB) dengan main-lobe sedikit lebih lebar, cocok untuk sinyal dengan dua komponen frekuensi yang berdekatan.
- **Flat-top:** window khusus untuk keakuratan amplitudo (amplitude accuracy); side-lobe paling rendah tetapi main-lobe paling lebar di antara keempatnya, sehingga cocok saat pembacaan amplitudo akurat lebih penting daripada resolusi frekuensi, misalnya kalibrasi sensor.

Gambar 6 memperlihatkan bentuk keempat window di domain waktu (panel atas) serta perbandingan langsung spectral leakage antara Rectangular dan Hann pada sebuah nada uji yang sengaja tidak persis berada di satu bin frekuensi (panel bawah): terlihat jelas Rectangular meninggalkan deretan side-lobe yang meluruh sangat lambat, sementara Hann meluruh jauh lebih cepat sehingga energi yang bocor ke bin-bin tetangga jauh lebih kecil.

Contoh Numerik: Mengapa Frekuensi Mesin Jarang Persis Bin-Aligned. Mengapa nada uji pada Gambar 6 sengaja dipilih tidak persis berada di satu bin frekuensi? Inilah inti masalah spectral leakage pada data lapangan: hampir tidak pernah frekuensi mesin sesungguhnya bertepatan persis dengan salah satu bin DFT. Sebagai contoh numerik konkret, misalkan motor beroperasi pada 1.780 rpm, setara $f_shaft = 1.780/60 = 29,67$ Hz, sedangkan resolusi frekuensi hasil akuisisi adalah $\Delta f = 1$ Hz (bin terletak tepat di 29 Hz dan 30 Hz, tidak ada bin persis di 29,67 Hz). Tanpa window (Rectangular), energi komponen

29,67 Hz akan bocor signifikan ke bin-bin tetangga (28, 29, 30, 31 Hz, dan seterusnya) mengikuti pola side-lobe yang meluruh lambat seperti tampak pada Gambar 6, sehingga estimasi amplitudo pada bin 30 Hz bisa keliru dan sulit dibedakan dari komponen 2x yang kebetulan berdekatan. Dengan window Hann, energi tetap "bocor" ke bin 29 dan 30 Hz karena posisi 29,67 Hz memang berada di antara keduanya, tetapi kontribusi ke bin-bin yang lebih jauh (28, 31, 32 Hz, dst.) tereduksi drastis mengikuti side-lobe -31 dB, sehingga puncak yang teramati jauh lebih tajam dan representatif terhadap frekuensi asli. Inilah alasan praktis mengapa vibration analyzer industri hampir selalu menerapkan windowing sebagai langkah standar sebelum FFT, bukan sekadar pilihan opsional, mengingat kondisi non-bin-aligned semacam ini adalah aturan, bukan pengecualian, pada data lapangan.



Gambar 6. Bentuk window function Rectangular, Hann, Hamming, dan Flat-top di domain waktu (atas), serta perbandingan spectral leakage Rectangular vs Hann pada nada uji tidak bin-aligned (bawah).

Amplitude Correction Factor dan Pemilihan Window: Karena window mengurangi energi total sinyal (mengalikan dengan bobot kurang dari atau sama dengan satu), magnitude FFT hasil windowing perlu dikoreksi dengan amplitude correction factor (ACF) agar kembali merepresentasikan amplitudo asli sinyal: untuk Hann, ACF sekitar 2,0; untuk

Hamming, ACF sekitar 1,85; untuk Flat-top, ACF sekitar 4,18. Library `scipy.signal.windows` menyediakan bentuk window sekaligus memudahkan perhitungan ACF melalui $ACF = 1/\text{mean}(\text{window})$. Pemilihan window yang tepat bergantung pada karakter sinyal: sinyal transient/impact memakai Rectangular karena sudah natural-zero di tepi; sinyal getaran rotor steady (motor, pompa, turbin) memakai Hann sebagai default umum; sinyal dengan frekuensi berdekatan (sidebands bearing fault) memakai Hamming; sedangkan kalibrasi/pengukuran amplitudo memakai Flat-top. Sebagian besar vibration analyzer industri menetapkan Hann sebagai default.

Tabel 2 merangkum perbandingan keempat window dari sisi side-lobe level, amplitude correction factor, dan use case yang paling sesuai, sebagai panduan cepat pemilihan window dalam praktik.

Tabel 2. Perbandingan empat window function terhadap side-lobe level, amplitude correction factor, dan use case.

Window	Side-lobe Level	ACF (Koreksi Amplitudo)	Use Case Utama
Rectangular	~ -13 dB	1,0 (tanpa koreksi)	Transient/impact test, sinyal natural-zero di tepi
Hann	~ -31 dB	~2,0	Default umum getaran rotor steady-state
Hamming	~ -43 dB	~1,85	Dua komponen frekuensi berdekatan (sidebands)
Flat-top	Terendah (main-lobe lebar)	~4,18	Kalibrasi sensor, akurasi amplitudo mutlak

ANALOGI KEHIDUPAN SEHARI-HARI

Enam analogi berikut menghubungkan konsep Transformasi Fourier dan FFT dengan pengalaman sehari-hari mahasiswa, mempermudah intuisi sebelum kembali ke rumus formal.

- **Lagu vs Partitur Musik:** sinyal domain waktu seperti rekaman lagu, kita hanya mendengar suara campur tanpa bisa memilah instrumen penyusunnya; spektrum domain frekuensi seperti partitur musik, kita melihat langsung not-not penyusunnya. Mendiagnosis kerusakan mesin tanpa FFT ibarat membaca lagu hanya dari rekaman waveform tanpa pernah melihat partiturnya.
- **Kipas Angin Tampak Diam di Bawah Lampu Neon:** baling-baling kipas angin yang berputar cepat di bawah lampu neon kadang tampak diam atau berputar sangat

lambat; ini terjadi karena lampu neon berkedip (flicker) pada frekuensi listrik (50/60 Hz) yang bertindak seperti "sample rate" mata kita, sementara putaran kipas melebihi setengah frekuensi kedip tersebut, persis mekanisme aliasing ketika f_s tidak memenuhi syarat Nyquist.

- **Prisma Memecah Cahaya Putih Menjadi Pelangi:** cahaya putih matahari yang dilewatkan prisma kaca terurai menjadi spektrum pelangi merah hingga ungu; prisma di sini berperan seperti Transformasi Fourier, mengurai satu berkas cahaya "campuran" (domain waktu) menjadi komponen-komponen warna/frekuensi tunggal penyusunnya (domain frekuensi).
- **Equalizer Audio Bass-Mid-Treble:** slider bass-mid-treble pada aplikasi pemutar musik langsung memanipulasi pita frekuensi tertentu dari lagu; ini adalah operasi domain frekuensi murni, mirip cara insinyur getaran fokus pada rentang frekuensi tertentu (misalnya di sekitar gear mesh frequency) untuk diagnosis, tanpa perlu mengubah keseluruhan sinyal domain waktu.
- **Vignette Halus di Tepi Foto Panorama:** saat menggabungkan beberapa foto menjadi panorama, tepi tiap foto diberi efek vignette/fade halus agar sambungannya tidak terlihat patah; ini persis fungsi window function, meratakan tepi buffer sinyal secara bertahap menuju nol agar FFT tidak "melihat" sambungan tepi yang tiba-tiba sebagai komponen frekuensi palsu (spectral leakage).
- **Radio Tuner Mencari Stasiun:** memutar knob radio analog perlahan-lahan untuk memisahkan dua stasiun yang frekuensinya berdekatan membutuhkan ketelitian putaran (resolusi) yang halus; ini analog langsung dengan resolusi frekuensi $\Delta f = f_s/N$ pada DFT/FFT, semakin halus Δf yang dibutuhkan (mis. memisahkan dua puncak berdekatan), semakin panjang waktu rekaman T yang harus disediakan.

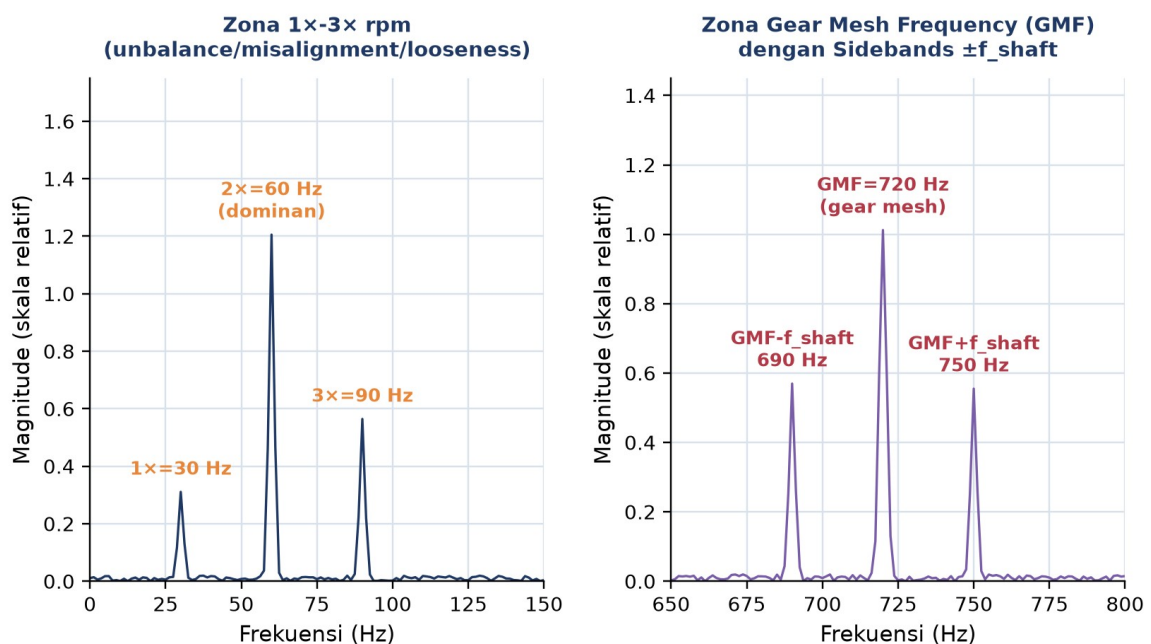
APLIKASI FFT DI INDUSTRI: STUDI KASUS DIAGNOSIS MOTOR DAN GEARBOX

Kasus Maintenance Pabrik: Motor Bising di Lantai Produksi. Tim maintenance pabrik menerima keluhan operator: sebuah motor induksi 1800 rpm yang menggerakkan gearbox rasio 1:3 (jumlah gigi $n_{\text{teeth}} = 24$) terdengar lebih bising dari biasanya, dengan getaran terasa pada housing. Accelerometer dipasang pada housing motor dengan sample rate $f_s = 5.120$ Hz selama 0,8 detik ($N = 4.096$ sample). Data domain waktu hanya memperlihatkan sinyal kompleks yang tidak periodik secara kasat mata; tim diminta melakukan FFT, mengidentifikasi puncak frekuensi karakteristik (1x, 2x, 3x rpm, gear mesh frequency

beserta sidebands), memvalidasi pemilihan f_s dan window, lalu memberikan diagnosis awal.

Setup Akuisisi dan Validasi Nyquist. Frekuensi shaft $f_{\text{shaft}} = \text{rpm}/60 = 1800/60 = 30$ Hz. Gear Mesh Frequency $\text{GMF} = n_{\text{teeth}} \times f_{\text{shaft}} = 24 \times 30 = 720$ Hz. Resolusi frekuensi $\Delta f = f_s/N = 5.120/4.096 = 1,25$ Hz, dan Nyquist frequency $= f_s/2 = 2.560$ Hz. Validasi setup: Nyquist 2.560 Hz jauh di atas GMF 720 Hz (faktor aman 3,6x), dan $\Delta f = 1,25$ Hz cukup halus untuk memisahkan harmonik $1x/2x/3x$ (berjarak 30 Hz, setara 24 bin) maupun sidebands di sekitar GMF (berjarak $f_{\text{shaft}} = 30$ Hz dari GMF). Durasi rekaman $T = N/f_s = 4.096/5.120 = 0,8$ detik, setara 24 putaran shaft penuh pada 1800 rpm, cukup untuk estimasi spektrum yang stabil. $N = 4.096 = 2^{12}$ juga merupakan pangkat dua, sehingga FFT radix-2 dapat bekerja optimal.

**Spektrum FFT Motor 1800 rpm + Gearbox 1:3 ($f_s=5120$ Hz, $N=4096$, $\Delta f=1,25$ Hz):
2x Dominan -> Misalignment; Sidebands GMF $\pm f_{\text{shaft}}$ -> Gear Wear**



Gambar 7. Spektrum FFT motor 1800 rpm plus gearbox rasio 1:3: zona 1x-3x rpm (kiri) menunjukkan 2x dominan sebagai indikasi misalignment; zona gear mesh frequency (kanan) menunjukkan GMF 720 Hz dengan sidebands GMF plus-minus f_{shaft} sebagai indikasi gear wear.

Interpretasi Spektrum dan Diagnosis. Gambar 7 menunjukkan hasil FFT lengkap dengan windowing Hann diterapkan. Pada zona 1x-3x rpm (panel kiri), puncak $1x=30$ Hz relatif kecil, sedangkan puncak $2x=60$ Hz jauh lebih dominan, sekitar empat kali amplitudo $1x$; menurut kaidah diagnosis standar, $1x$ dominan menandakan unbalance sementara $2x$ dominan menandakan misalignment antara poros motor dan gearbox. Pada zona gear mesh frequency (panel kanan), puncak $\text{GMF}=720$ Hz didampingi dua sidebands simetris pada $\text{GMF}-f_{\text{shaft}}=690$ Hz dan $\text{GMF}+f_{\text{shaft}}=750$ Hz; pola sidebands simetris di sekitar

GMF ini adalah tanda khas gear wear atau eksentrisitas roda gigi, karena satu gigi yang aus/tidak seragam akan memodulasi amplitudo gear mesh sekali setiap putaran shaft. Diagnosis gabungan yang disarankan: lakukan laser alignment pada kopling motor-gearbox sebagai langkah pertama (mengatasi sumber 2x dominan), kemudian inspeksi visual kondisi permukaan gigi gearbox (mengatasi sumber sidebands GMF).

Peringatan: Verifikasi Sumber Elektrik vs Mekanik. Satu langkah validasi yang sering terlewat: sebelum menyimpulkan sumber mekanik untuk setiap puncak, periksa dahulu apakah puncak tersebut berimpit dengan frekuensi listrik jala-jala (di Indonesia, 50 Hz dan kelipatannya seperti 100 Hz), karena motor induksi kerap memunculkan getaran elektromagnetik pada f_{line} dan $2 \times f_{line}$ akibat torsional ripple atau eksentrisitas rotor-stator. Tes diagnostik standar adalah mematikan motor sesaat dan mengukur sinyal "latar" (background); bila puncak yang dicurigai hilang saat motor mati, sumbernya elektrik, bukan mekanik. Pada kasus ini, puncak 60 Hz bertepatan dengan 2x rpm ($30 \text{ Hz} \times 2 = 60 \text{ Hz}$) dan bukan kelipatan 50 Hz jala-jala Indonesia, sehingga kemungkinan besar sumbernya memang mekanik (misalignment), namun langkah verifikasi baseline tetap disarankan sebagai praktik baik sebelum mengeksekusi rencana perbaikan.

Aplikasi Fourier di Luar Vibration Analysis: Di luar diagnosis getaran, Transformasi Fourier dan turunannya menopang banyak teknologi Teknik Mesin dan bidang terkait lainnya: analisis modal eksperimental memakai FFT untuk mengekstrak frekuensi pribadi dari data uji impact hammer; sistem kontrol getaran aktif pada mesin CNC memakai FFT real-time untuk mendeteksi chatter; balancing rotor dinamis memakai magnitude dan fase komponen 1x hasil FFT untuk menghitung koreksi massa; bahkan pemrosesan sinyal akustik untuk deteksi kebocoran pipa bertekanan juga mengandalkan analisis spektrum FFT yang sama prinsipnya dengan yang dipelajari pada modul ini.

IMPLEMENTASI PYTHON DI JUPYTER NOTEBOOK

Empat cell Python berikut mengimplementasikan workflow FFT lengkap dan runnable langsung di VS Code Jupyter atau Google Colab, konsisten dengan studi kasus dan rumus yang telah dibahas.

- **Cell 1 - Generate Sinyal Sintetis:** membangkitkan sinyal sintetis tiga komponen frekuensi (50 Hz amplitudo 1,0; 120 Hz amplitudo 0,5; 350 Hz amplitudo 0,3) plus noise putih $\sigma=0,1$, pada $f_s = 2.000 \text{ Hz}$ selama $T = 2 \text{ detik}$ ($N = 4.000 \text{ sample}$); mencetak sample rate, jumlah sample, resolusi frekuensi f_s/N , dan Nyquist frequency $f_s/2$ sebagai sanity check sebelum FFT dijalankan.

- **Cell 2 - FFT Dasar dengan `numpy.fft.rfft`:** menghitung FFT dasar dengan `numpy.fft.rfft(x)` (efisien untuk sinyal real, hanya menghitung setengah spektrum), memperoleh sumbu frekuensi dengan `numpy.fft.rfftfreq(N, d=1/fs)`, lalu menormalisasi magnitude dengan $\text{mag} = |X| \times 2/N$ (bin DC dibagi 2 karena tidak digandakan), dan mencetak tiga puncak tertinggi memakai `np.argsort`.
- **Cell 3 - Windowing dengan Hann dan Correction Factor:** menerapkan Hann window dari `scipy.signal.windows.hann(N)`, menghitung amplitude correction factor sebagai $1/\text{mean}(\text{window})$ (bernilai sekitar 2,0 untuk Hann), lalu membandingkan magnitude pada frekuensi 50 Hz sebelum dan sesudah windowing untuk mendemonstrasikan efek koreksi amplitudo dan reduksi spectral leakage.
- **Cell 4 - Plot Spektrum dan Deteksi Puncak Otomatis:** memplot dua panel (waveform domain waktu dan spektrum magnitude domain frekuensi) memakai `matplotlib`, lalu mendeteksi puncak otomatis dengan `scipy.signal.find_peaks` (height dan distance threshold), memberi anotasi frekuensi tiap puncak yang terdeteksi langsung pada grafik spektrum.

Gambar 8 menunjukkan potongan Cell 2, langkah inti workflow: menghitung FFT real-input dengan `numpy.fft.rfft`, menormalisasi magnitude ke satuan amplitudo asli, dan mencetak tiga puncak frekuensi tertinggi hasil pengurutan dengan `np.argsort`. Untuk data real dari accelerometer (format CSV atau TDMS), Cell 1 dapat digantikan dengan `pd.read_csv()` atau library `nptdms`, sementara workflow Cell 2 sampai Cell 4 tetap sama persis.

Verifikasi Output: Nilai yang Diharapkan dari Cell 1 dan Cell 2. Bila Cell 1 dan Cell 2 dijalankan berurutan pada dataset referensi ($f_s = 2.000$ Hz, $N = 4.000$, komponen 50 Hz amplitudo 1,0; 120 Hz amplitudo 0,5; 350 Hz amplitudo 0,3 plus noise $\sigma=0,1$), output `print()` yang diharapkan pada Cell 1 adalah "Sample rate: 2000.0 Hz", "Total samples: 4000", "Frequency resolution: 0.5 Hz/bin", dan "Nyquist: 1000.0 Hz"; sedangkan Cell 2 akan mencetak "FFT output length: 2001", "Frequency range: 0 - 1000.0 Hz", diikuti tiga baris "Top 3 peaks" yang mendekati $f = 50,00$ Hz dengan amplitudo sekitar 1,000, $f = 120,00$ Hz dengan amplitudo sekitar 0,500, dan $f = 350,00$ Hz dengan amplitudo sekitar 0,300, konsisten dengan amplitudo asli yang didefinisikan pada Cell 1 (nilai persisnya dapat sedikit bergeser akibat kontribusi noise acak). Mahasiswa disarankan menjalankan sendiri kedua cell ini di Jupyter Notebook dan membandingkan hasil cetak tersebut sebagai langkah verifikasi bahwa implementasi `numpy.fft` berjalan benar, sebelum melangkah ke Cell 3 (windowing) dan Cell 4 (deteksi puncak otomatis) yang menjadi dasar pengerjaan sebagian besar soal Tugas Pertemuan 12 pada bagian berikutnya.

```
cell2_fft_dasar.py

# Real-input FFT (efisien utk sinyal real)
X = np.fft.rfft(x)
freqs = np.fft.rfftfreq(N, d=1/fs)

# Magnitude -- normalisasi 2/N
magnitude = np.abs(X) * 2 / N
magnitude[0] /= 2 # bin DC

top_idx = np.argsort(magnitude)[-3:][::-1]
for i in top_idx:
    print(f"f={freqs[i]:6.2f} Hz |
          amp={magnitude[i]:.3f}")
```

Gambar 8. Potongan Cell 2: FFT dasar dengan `numpy.fft.rfft`, normalisasi magnitude ke satuan amplitudo asli, dan pencarian tiga puncak frekuensi tertinggi.

TUGAS PERTEMUAN 12

Tugas terdiri atas 10 soal Pilihan Ganda (1 poin), 10 soal Komputasi Easy-Medium (2 poin), dan 5 soal Komputasi Hard (4 poin), total 50 poin, seputar transformasi Fourier (DFT/FFT), frekuensi Nyquist dan aliasing, resolusi frekuensi, windowing dan spectral leakage, normalisasi amplitudo, serta interpretasi spektrum getaran mesin. Gambar 9 merangkum distribusi bobotnya.

Petunjuk Pengerjaan dan Submit Tugas. Seluruh soal komputasi (Bagian B dan Bagian C) dikerjakan di Jupyter Notebook (VS Code) dengan bantuan numpy dan scipy. Setelah kode diuji dan berjalan benar di Jupyter, mahasiswa menempelkan (paste) kode tersebut ke kolom yang tersedia pada halaman modul interaktif, lalu menekan tombol "Run & Validasi"; sistem akan menjalankan kode di browser melalui Python WebAssembly, menangkap output `print()`, dan mencocokkan nilai numerik yang tercetak dengan jawaban yang diharapkan. Kode wajib menyertakan baris import yang relevan (misalnya import `numpy as np`) agar dapat dieksekusi tanpa error. Di akhir pengerjaan, mahasiswa menempelkan satu tautan Google Drive (dengan akses "Anyone with the link") yang berisi file Jupyter Notebook (.ipynb) lengkap, mencakup baik kode implementasi Python pada Bagian Implementasi Python di atas maupun jawaban lengkap seluruh soal tugas. Langkah

terakhir adalah menekan tombol "Export HTML" untuk mengunduh berkas hasil pengerjaan, yang kemudian disubmit ke Fast Learning (LMS UMB) pada menu Tugas Pertemuan 12 sesuai tenggat waktu yang telah ditetapkan dosen.

Distribusi Bobot Tugas Pertemuan 12



Gambar 9. Distribusi 50 poin Tugas Pertemuan 12 berdasarkan tipe soal.

Bagian A: Pilihan Ganda (10 soal x 1 poin)

1. Apa fungsi utama Transformasi Fourier dalam analisis getaran mesin?

- A. Mengukur amplitudo getaran dalam time domain
- B. Mengubah sinyal time-domain menjadi spektrum frequency-domain untuk identifikasi komponen frekuensi
- C. Menghilangkan noise dari sinyal getaran
- D. Mengintegrasikan acceleration menjadi velocity

2. Untuk sample rate $f_s = 2000$ Hz, berapa frekuensi Nyquist (frekuensi maksimum yang bisa direkam tanpa aliasing)?

- A. 500 Hz
- B. 1000 Hz
- C. 2000 Hz
- D. 4000 Hz

3. Apa perbedaan utama antara DFT (Discrete Fourier Transform) dan FFT (Fast Fourier Transform)?

- A. DFT untuk sinyal continuous, FFT untuk discrete
- B. DFT lebih akurat dari FFT
- C. Hasil sama, FFT adalah algoritma cepat untuk hitung DFT ($O(N \log N)$ vs $O(N^2)$)
- D. FFT hanya bekerja untuk sinyal periodik, DFT untuk semua sinyal

4. Kondisi yang menyebabkan aliasing terjadi adalah...

- A. Sample rate terlalu tinggi (oversampling)
- B. Sinyal mengandung komponen frekuensi di atas $f_s/2$ (Nyquist) tanpa anti-aliasing filter
- C. Menggunakan window function
- D. Sinyal tidak periodik

5. Untuk $f_s = 2000$ Hz dan jumlah sample $N = 4000$, berapa resolusi frekuensi Δf per bin?

- A. 0,5 Hz
- B. 1,0 Hz
- C. 2,0 Hz
- D. 4,0 Hz

6. Apa tujuan utama menggunakan window function (Hann, Hamming) sebelum FFT?

- A. Memperbesar amplitudo sinyal
- B. Mengurangi spectral leakage karena sinyal tidak tepat periodik dalam buffer
- C. Mengubah sample rate sinyal
- D. Menghilangkan komponen DC

7. Sebuah peak tinggi muncul di FFT spectrum pada frekuensi 30 Hz untuk getaran motor 1800 rpm. Komponen ini adalah...

- A. Komponen 1x rpm (rotor unbalance kandidat utama)
- B. BPFO bearing fault
- C. Frekuensi mesh gear
- D. Aliasing dari frekuensi tinggi

8. Untuk mendapat magnitude FFT dengan satuan amplitudo asli, output `np.fft.rfft(x)` harus dikalikan dengan...

- A. $1/N$
- B. $2/N$ (untuk bin selain DC dan Nyquist)
- C. N
- D. $2/\text{akar}(N)$

9. Untuk sinyal real (bukan complex), output FFT memiliki properti symmetry. Berapa jumlah bin independen dari output `np.fft.fft(x)` dengan N sample?

- A. N
- B. $N/2$
- C. $N/2 + 1$
- D. $2N$

10. Kapan analisis frequency-domain (FFT) lebih informatif dibanding time-domain?

- A. Saat sinyal hanya 1 komponen sinusoidal sederhana

- B. Saat ingin melihat shock/transient response
- C. Saat sinyal mengandung banyak komponen frekuensi tercampur (mesin running steady-state)
- D. Saat sample rate sangat rendah

Bagian B: Komputasi Easy-Medium (10 soal x 2 poin)

Setiap soal mencantumkan parameter sinyalnya sendiri (f_s , N , dan frekuensi komponen). Gunakan numpy/scipy (`np.fft.rfft`, `np.fft.rfftfreq`, `scipy.signal`) sesuai kebutuhan. Lihat Bagian Implementasi Python untuk template kode.

C1. Hitung frekuensi Nyquist (Hz) untuk sample rate $f_s = 2000$ Hz. Formula: $f_{\text{Nyquist}} = f_s/2$.

- **HINT:** Frekuensi Nyquist = $f_s/2$; bagi sample rate dengan dua.

C2. Hitung resolusi frekuensi Δf (Hz/bin) untuk $f_s = 2000$ Hz dan $N = 4000$ sample.

- **HINT:** Resolusi frekuensi $\Delta f = f_s/N$; bagi sample rate dengan jumlah sampel.

C3. Generate sinyal $x = \sin(2\pi 50 t)$ dengan $f_s = 1000$ Hz dan durasi 1 detik. Hitung max amplitude (peak value).

- **HINT:** Bangun vektor waktu, bentuk sinyal dengan `np.sin`, lalu ambil nilai puncak dengan `np.max`.

C4. Generate $x = \sin(2\pi 100 t)$ dengan $f_s = 1000$, $N = 1000$. Lakukan FFT dan return frekuensi peak (Hz) dari spektrum.

- **HINT:** Hitung spektrum dengan `np.fft.rfft` dan sumbu frekuensi `np.fft.rfftfreq`, lalu cari indeks magnitudo terbesar dengan `np.argmax`.

C5. Untuk $f_s = 1000$ Hz dan $N = 1000$, hitung indeks bin (k) yang berkaitan dengan frekuensi 50 Hz. Formula: $k = \text{freq} \times N/f_s$.

- **HINT:** Indeks bin $k = \text{freq} \times N/f_s$; substitusikan frekuensi target, N , dan f_s lalu bulatkan ke integer.

C6. Hitung magnitude FFT di bin frekuensi 50 Hz untuk $x = \sin(2\pi 50 t)$, $f_s = 1000$, $N = 1000$. Magnitude = $|X[k]| \times 2/N$.

- **HINT:** Hitung `np.fft.rfft`, ambil magnitudo $|X| \times 2/N$, lalu baca nilai pada bin frekuensi target.

C7. Generate sinyal multi-component: $x = \sin(2\pi 50 t) + 0,5 \sin(2\pi 120 t)$, $f_s = 1000$, $N = 1000$. Lakukan FFT, return frekuensi peak tertinggi kedua (setelah sorting magnitude descending).

- **HINT:** Hitung magnitudo FFT (`np.fft.rfft`), urutkan menurun dengan `np.argsort`, lalu baca frekuensi pada dua puncak teratas.

C8. Hitung amplitude correction factor untuk Hann window dengan $N = 1000$. Formula: $1/\text{np.mean}(\text{window})$.

- **HINT:** Bangun window dengan `scipy.signal.windows.hann`, lalu amplitude correction factor = $1/\text{mean}(\text{window})$ (`np.mean`).

C9. Apply `scipy.signal.windows.hann(1000)` sebagai window, hitung dan print nilai maksimum dari window tersebut.

- **HINT:** Bentuk Hann window dengan `scipy.signal.windows.hann`, lalu ambil nilai maksimum dengan `np.max` (puncak di tengah).

C10. Hitung RMS dari sinyal sinusoidal pure $x = \sin(2\pi 100 t)$, $f_s = 1000$, $N = 1000$. RMS = $\text{akar}(\text{mean}(x^2))$.

- **HINT:** RMS = $\text{akar}(\text{mean}(x^2))$; pakai `np.mean` pada kuadrat sinyal lalu `np.sqrt`.

Bagian C: Komputasi Hard (5 soal x 4 poin)

Setiap soal membutuhkan implementasi algoritma lengkap (15-40 baris kode), bukan sekadar substitusi formula, dengan fokus pada komputasi DFT/FFT, filtering dan PSD, deteksi peak, serta analisis multi-konsep. Partial credit +1 poin diberikan bila kode non-kosong disubmit meski output belum benar atau terjadi runtime error.

C11 (Hard). Implementasikan DFT manual untuk $N = 8$ sample dari signal $x = [1, 0, 1, 0, 1, 0, 1, 0]$. Hitung $X[0]$ menggunakan formula DFT: $X[k] = \sum x[n] \exp(-j 2\pi k n/N)$. Verifikasi dengan `np.fft`. Return $|X[0]|$ (= sum of x).

- **HINT:** Terapkan formula DFT $X[k] = \sum x[n] \exp(-j 2\pi k n/N)$; ingat untuk $k=0$ faktor eksponensial = 1 sehingga $X[0] = \text{jumlah seluruh } x[n]$.

C12 (Hard). Apply bandpass filter (cutoff 30-70 Hz, Butterworth order 4) ke sinyal $x = \sin(2\pi 50 t) + \sin(2\pi 150 t)$ ($f_s=1000$, $N=1000$). Return RMS dari sinyal yang sudah difilter (komponen 50 Hz harus survive, 150 Hz dibuang).

- **HINT:** Rancang bandpass Butterworth order 4 dengan `scipy.signal.butter` (cutoff dinormalisasi ke Nyquist), terapkan `filtfilt`, lalu hitung RMS sinyal hasil filter.

C13 (Hard). Auto-detect 3 peaks tertinggi di FFT spektrum dari signal $x = \sin(2\pi 50 t) + 0,7 \sin(2\pi 120 t) + 0,4 \sin(2\pi 230 t)$ ($f_s=1000$, $N=2000$). Return frekuensi peak tertinggi (yang harusnya 50 Hz). Pakai `scipy.signal.find_peaks`.

- **HINT:** Hitung magnitudo FFT, deteksi puncak dengan `scipy.signal.find_peaks`, lalu ambil frekuensi pada puncak dengan magnitudo tertinggi.

C14 (Hard). Hitung Power Spectral Density (PSD) dengan Welch method untuk signal $x = \sin(2\pi 100 t) + 0,5 \text{ noise}$ ($f_s=1000$, $N=4000$, $\text{sigma noise}=0,3$). Return frekuensi peak dominan dari PSD.

- **HINT:** Hitung PSD dengan `scipy.signal.welch`, lalu cari frekuensi pada puncak PSD dengan `np.argmax`.

C15 (Hard). Generate chirp signal dengan frekuensi naik 50 sampai 200 Hz selama 2 detik ($f_s=1000$, $N=2000$). Hitung FFT magnitude maksimum (peak amplitude di mana saja di spektrum). Pakai `scipy.signal.chirp`.

- **HINT:** Bangun chirp dengan `scipy.signal.chirp`, hitung magnitudo FFT ($|X| \times 2/N$), lalu ambil nilai maksimum dengan `np.max`; energi chirp tersebar sehingga puncaknya rendah.

DAFTAR PUSTAKA

- Ghazali, M. H. M., & Rahiman, W. (2021). Vibration analysis for machine monitoring and diagnosis: A systematic review. *Shock and Vibration*, 2021, Article 9469318. <https://doi.org/10.1155/2021/9469318>
- Hassan, I. U., Panduru, K., & Walsh, J. (2024). An in-depth study of vibration sensors for condition monitoring. *Sensors*, 24(3), 740. <https://doi.org/10.3390/s24030740>
- Heo, J., Jung, Y., Lee, S., & Jung, Y. (2021). FPGA implementation of an efficient FFT processor for FMCW radar signal processing. *Sensors*, 21(19), 6443. <https://doi.org/10.3390/s21196443>
- Huang, S., Hong, M., Lin, G., Tang, B., & Shen, S. (2025). A discrete Fourier transform-based signal processing method for an eddy current detection sensor. *Sensors*, 25(9), 2686. <https://doi.org/10.3390/s25092686>
- Jwo, D.-J., Chang, W.-Y., & Wu, I.-H. (2021). Windowing techniques, the Welch method for improvement of power spectrum estimation. *Computers, Materials & Continua*, 67(3), 3983-4003. <https://doi.org/10.32604/cmc.2021.014752>
- Zhou, P., Yu, X., Yang, Y., He, Q., & Peng, Z. (2025). Fault-induced gear meshing modulation sideband extraction and evaluation for fault diagnosis of planetary gearboxes. *Science China Technological Sciences*, 68, Article 1120305. <https://doi.org/10.1007/s11431-024-2802-y>